

Decision Models at the Game Table: Where Non-Generative Classifiers Help and Fail in LLM Tooling for Tabletop Role-Playing Campaigns

[Author name]
[Affiliation / independent researcher]
[contact email]

October 2026

Abstract

Decision models are a recent class of non-generative language models: a backbone reads a *state* once and a small head returns a probability for every option of every typed question (multiple choice, yes/no, ordinal score), with no text generated. Two open families appeared in October 2026, Cloudflare’s Clef and vLLM Semantic Router’s Decision 2.0, both promising fast, calibrated classification. We test whether they can replace or assist a generative model in the judgment steps of a working LLM pipeline for tabletop role-playing campaigns, served on a local NVIDIA DGX Spark (GB10). We use labels taken from the game master’s own past rulings: 525 reviewed transcript-correction cards, 864 triaged tokens, 335 spelling rulings, and 115 entity-type merge decisions, plus 12 paired NPC-behaviour scenarios. Decision models work where the label is a coarse property readable from meaning: Clef 27B forwarded only 37–56% of a transcript’s unknown-token pile without discarding any of 84 real names, and four decision models moved NPC behaviour in the expected direction on 11 of 12 paired scenarios against 5 of 12 for a published dice-table system. They fail where the label is a string-level judgment (choosing a canonical spelling: 66% accuracy with 71 confident wrong answers, against 94.5% of answered items for a local generative model with reasoning enabled), a house convention, or a consistency property across many items (typing extracted facts: re-typing per fact created about as many split entities as it repaired). We also report that an apparently strong verdict-prediction result (AUC 0.89) was an artifact of the classifier reading the upstream model’s own hedging, and that vendor latency claims did not transfer to the GB10 (127 ms measured vs. 18.4 ms claimed).

1 Introduction

Large language model pipelines for long-running creative work make many small judgment calls: is this token a misheard proper noun, which entity does this fact describe, how would this character react. In a pipeline that turns recorded game sessions into campaign records, each of these calls feeds the next step, and an error compounds silently unless a human checkpoint catches it. A practitioner therefore wants two things from the model making those calls: it should be right, and it should *know when it is not*, so that uncertain cases can be routed to the human instead of being decided.

Decision models address both wants directly. Instead of generating an answer, they score a fixed set of options and return a probability for each, in one forward pass. Two open families were released within days of each other in October 2026: Cloudflare’s *Clef* (27B and 9B backbones) [1]

and vLLM Semantic Router’s *Decision 2.0* (0.6B to 27B) [2], both exposing the same *SystemOne* request format and both advertising low latency.

This paper reports a practitioner’s evaluation of these models against real workloads rather than benchmarks. The workloads come from an open-source campaign-management toolkit used to run several tabletop role-playing campaigns; the labels come from one game master’s (GM’s) recorded rulings on earlier model output. All models ran on local hardware (two NVIDIA DGX Spark systems, GB10, 128 GB unified memory each), deliberately: part of the aim was to calibrate what local serving of such models is like in practice.

We ask four questions:

RQ1 Can a decision model replace a generative model in the *choice* steps of the pipeline (spelling selection, entity typing)?

RQ2 Can it act as a cheap *filter* or *flag* in front of an expensive model or a human reviewer?

RQ3 Are its probabilities useful for *sampling* plausible behaviour (NPC reactions), where a distribution matters more than an argmax?

RQ4 Do the published speed claims hold on this hardware?

Contributions. (i) Results on five tasks with labels taken from a GM’s own rulings, including two strong negative results; (ii) a characterization of *which kinds of label* decision models can and cannot learn from the input; (iii) two methodological pitfalls that produced convincing but spurious results in this study (label leakage through upstream prose; selection bias in a repair-only test set); (iv) deployment notes for the GB10, including a unified-memory accounting bug in an upstream runtime.

2 Background

2.1 Decision models and the SystemOne interface

A request is a **state** (text) and a dictionary of named **questions**, each typed **choice** (a dictionary of option keys to descriptions), **noul** (yes/no) or **score** (an ordered list of levels). The model prefills the state and the question text once; a small head scores every option of every question jointly. The response carries a probability per option. Clef’s release implements the head as a separate set of weights (“joint schema head”) on Qwen-family backbones [1]. Decision 2.0 is served by a standalone runtime in vLLM Semantic Router [4, 5] that exposes `POST /v1/decisions`, a superset of SystemOne, with `/v1/systemone` as an alias [3]. Because the two families share the interface, every experiment here ran against either family by changing only the endpoint.

2.2 The workloads

The toolkit’s pipeline turns a session’s audio transcript into a reviewed transcript, a structured summary, scene extractions and finally narrative prose. Three steps involve classification-shaped judgments:

- **Transcript spelling pass.** Automatic speech recognition mangles fantasy names (*Zalthir* → “Zalfir”). A pass collects unknown capitalised tokens, filters obvious non-names, proposes a canonical spelling for the rest, and asks the GM to approve each proposal on a review page (Approve / Reject / Discuss).

Model	Family	Backbone	Measured median latency (GB10)
clef	Clef	Qwen3.8-27B	400–750 ms
clef-flash	Clef	Qwen3.5-9B	130–250 ms
Decision-2.0-Lux-9B	Decision 2.0	Qwen3.5-9B	127 ms (claimed 18.4 ms)
Decision-2.0-Nox-4B	Decision 2.0	Qwen3.5 (4B)	80 ms
Decision-2.0-Kai-0.6B	Decision 2.0	Qwen3-0.6B	16 ms (claimed 4.9 ms)
qwen3.8-flash-next, fast	generative	—	2.0–3.8 s
qwen3.8-flash-next, thinking	generative	—	≈33 s (answered items)

Table 1: Models and measured per-request latency for the request shapes used here (one request, one to two questions, a few hundred to ~2,000 input tokens). Ranges reflect different tasks. Claimed latencies are from the model cards, measured on an unnamed GPU.

- **Ensemble fact extraction.** Five extraction passes over each chapter emit facts with a **type** (npc, monster, faction, location, object, event, thread, date). Downstream, facts are grouped into per-entity dossiers keyed by (type, canonical subject).
- **NPC behaviour at the table.** During play the GM must decide how a non-player character responds. A published system, FlexAI [6], does this with d100 tables keyed on the creature’s role and stance; it never sees the situation itself.

2.3 The human-checkpoint principle

The toolkit’s design rule is that a model may draft, but *precision decisions* (scope, ordering, attribution, canonical spellings) require a human checkpoint before they feed another model. This shapes what counts as a useful classifier: a model that cannot be trusted to decide may still be valuable if it reliably *orders* or *flags* work for the human, provided nothing is dropped silently.

3 Setup

3.1 Models and serving

Table 1 lists the models. Clef and clef-flash were served by a thin FastAPI wrapper around the release’s own model code using PyTorch 2.11 (CUDA 13.0); the release’s suggested vLLM command would load only the chat backbone and silently drop the decision head. Decision 2.0 was served by vllm-sr-runtime [3] at a pinned commit, one model per container. The generative baseline was qwen3.8-flash-next (NVFP4/FP8 hybrid quantisation, served by vLLM) with reasoning either disabled (“fast”) or enabled with a 4,000-token budget (“thinking”).

3.2 Labels

All labels are rulings the GM made on earlier model output in one campaign (an adapted *Out of the Abyss*). Four sources are used:

1. **525 transcript-correction review cards** across 42 sessions: 438 approved, 82 discussed, 4 ruled table chatter, 1 rejected.
2. **864 triaged tokens** from the nine sessions whose review pages enumerated what was left alone: 628 left alone, 152 ruled table chatter, 84 that became approved name-correction cards.

Card subset	n (not approved)	AUC, upstream prose	AUC, raw sources
All cards	525 (87)	0.887	0.911
Proposes a fix	418 (10)	0.665	0.658
Open question, no fix	107 (77)	0.800	0.398
Baseline: “has a fix”	525 (87)	0.908	

Table 2: E1, clef 27B predicting the GM’s verdict. The headline AUC comes from card type and from upstream prose, not from judgment about the correction itself.

“Left alone” means the GM saw the token listed and did not pull it back, a weaker label than a card ruling.

3. **335 spelling items:** 323 GM-approved name corrections whose canonical form exists in the campaign’s name lists, plus 12 ruled table chatter (correct answer: leave as spoken).
4. **115 entity-type merge rulings** over ensemble dossiers: 109 merged (with a GM-chosen primary type) and 9 kept separate (some groups mix outcomes).

The campaign text is not released: it quotes play sessions verbatim and the table-chatter labels name real people. The evaluation scripts are public [9].

3.3 Metrics

For verdict prediction we report ROC AUC [7]. For filters we report, at each threshold, the fraction of items forwarded and the number of real names *missed*, since in this pipeline a filtered-out real name is a silent deletion that nothing downstream can recover. For choice tasks we report accuracy and separately count *confident wrong names*, the error a tired reviewer approves.

4 Experiments and results

4.1 E1: Predicting the GM’s verdict (and how it fooled us)

Task. For each of the 525 review cards, ask “Will the GM approve this proposed correction as written?” (noul). If this worked, the review page could be sorted so the GM looks hardest at the likely rejections.

Result. On the cards as the upstream model wrote them, clef reached AUC 0.887 and clef-flash 0.815. Rebuilding each card from raw sources only (transcript lines, the second transcription, a canon lookup) raised clef to 0.911.

Why it is not a result. A one-feature baseline, “does the card propose a fix at all”, scores 0.908: 77 of the 107 cards with no proposed fix were not approved, against 10 of 418 cards with one. Splitting by card type (Table 2) shows that on open questions the strong score existed only when the classifier could read the upstream model’s prose, and fell *below chance* (0.398) without it. The classifier was reading the drafting model’s hedges (“could be X or Y”, “what was said?”) back to us. On proposals, the verdict is saturated (97.6% approved) and the 10 non-approvals are too few to measure.

Model	Accuracy	Confident wrong names	Unanswered	Median latency
clef-flash	30.4%	52	0	0.25 s
clef	66.0%	71	0	0.75 s
qwen, fast	75.8%	56	0	3.8 s
qwen, thinking	62.1% (94.5% of 220 answered)	7	115	≈33 s answered
clef $\wedge$ qwen-thinking agree	95.8% of 166	5	—	—

Table 3: E2, 335 items. “Unanswered” means the reasoning budget ran out; 51 of the 65 items whose correct name was not in the candidate list fall here, so exhaustion behaves as a safe abstention that routes the item to the GM. Clef’s confidence was weakly informative (its most confident half was 77.8% correct); the fast generative model’s stated confidence was not (79.6%).

4.2 E2: Choosing the canonical spelling

Task. Given a transcript token, about 21 cues of raw transcript around it and the second transcription’s reading, pick the canonical spelling from a candidate list: the eight nearest names by spelling and Double Metaphone [8] similarity, plus OTHER (a real name not in the list) and LEAVE (not a misheard name). Candidates came from canonical names only; the glossary’s wrong-form column was excluded because it contains these very rulings.

Candidate recall caps every model. The correct name was among the eight candidates for 258 of 323 items (80%). The misses are mostly cases where neither spelling nor sound is close enough to retrieve the name (*Vira* $\rightarrow$ Ilvara, *Peto* $\rightarrow$ Buppido), plus a house convention of transcribing the GM’s own name as “GM”.

Instruction sensitivity. With the question phrased as “which name was spoken”, both models tried on that wording (clef-flash and the fast generative model) expanded a spoken surname into a full name (*Misrim* $\rightarrow$ “Ilvara Mizzrym”), which would insert words nobody said. Rephrased as “the correct spelling of exactly the words spoken”, the generative model complied and clef did not.

Result (Table 3). Decision models lose clearly. Gating the generative model on clef’s confidence added nothing. Note the test set’s bias: every item is a proposal the upstream model made and the GM approved, so it measures how close local models come to approved work and cannot reveal the upstream model’s own misses.

4.3 E3: Triage of the unknown-token pile

Task. Before any expensive judgment, classify each capitalised unknown token as filler, rules term, real-world chatter, or campaign name (**choice**), and forward it only if $P(\text{campaign})$ clears a threshold. Nothing is deleted: tokens below the threshold still appear in the review page’s “left alone” list.

Result (Table 4). This is the task where decision models earned a place: clef 27B roughly halves the expensive step’s input without a single silent loss, and survives a held-out split. Smaller decision models trade that guarantee for speed; the names they drop recur across models (e.g. *Heel Strike*).

4.4 E4: Entity typing in ensemble extraction

Per entity. For each of the 92 usable merged groups, the classifier saw a sample of the entity’s raw facts with the original type labels hidden. A free majority vote over the extraction passes’ own types matched the GM’s primary on 82/92; clef matched 82, clef-flash 80, the fast generative model 80. Voting with the model deciding only when no type holds 60% of facts scored 87 for *all three*

Model	Forwarded, 0 of 84 missed	Forwarded, ≤ 2 missed	Chatter recognised	Median
clef	37–56%	—	136/152	400 ms
clef-flash	98%	66%	128/152	132 ms
Decision-2.0-Lux-9B	89%	54–71%	129/152	127 ms
Decision-2.0-Nox-4B	96%	36–72%	118/152	80 ms
Decision-2.0-Kai-0.6B	98%	78%	7/152	16 ms
qwen, fast	—	24–38% (5–6 missed)	146/152	2.0 s

Table 4: E3, 864 tokens, 84 of them real names. “Forwarded” is the share of tokens sent on to the expensive step at a threshold achieving the stated number of misses; ranges span the thresholds that achieve it. For clef, a threshold tuned on half the sessions (then halved for margin) forwarded 44% and 51% of the held-out halves with 0 misses.

Who types each fact	Split entities (of 92 merged)	New splits (of 55 consistent)	Facts moved to non-entity types
extraction (as is)	78	0 (by selection)	—
clef	29	7	255 + 177
clef-flash	46	13	120 + 68
qwen, fast	38	13	28 + 19

Table 5: E4, per-fact re-typing. “Facts moved” gives repair set + control set. Projected over the campaign’s ~ 470 consistent dossiers, re-typing creates roughly as many splits as it repairs (clef) or more (the others).

models. Most disagreements are filing conventions that the facts do not reveal: the GM filed the demon lords Zugtmoy, Juiblex and Demogorgon as *monster* but Yeenoghu, also a demon lord, as *npc*; the campaign’s own entity registry types all four *npc*. These are canon questions for the GM, not model errors.

Per fact. The idea tested was to let the generative model extract facts and a decision model assign each fact’s type. On the 1,691 facts of those entities, extraction had typed 266 with a different entity type, splitting 78 entities. Re-typing reduced splits (Table 5), but this set was *selected on extraction’s mistakes*, so any second opinion looks good on it. On a control set of 55 entities that extraction had typed consistently (847 facts), every re-typer created new splits, and clef moved 21% of facts into non-entity types (*event*, *thread*, *date*), removing them from their dossiers.

As an extra signal. Adding decision-model votes to the majority vote hurt (81/92 voting over clef’s per-fact types; 82/92 pooled; 84/92 for extraction alone on this fact set). Used instead as a *second opinion* that flags subjects where it disagrees with the vote, clef flagged 15/92 and caught 8 of the vote’s 10 errors (7 false alarms); the fast generative model (17 flags, 8/10) and clef-flash (18, 8/10) did nearly as well.

Interpretation. The failure is structural. Per-fact typing gives an entity with dozens of facts dozens of independent chances to draw one odd label, and one odd label is a split. A deterministic model guarantees the same answer for the same input, but every fact is a different input. The repair is to type once per entity, which needs no model; we have proposed that design separately.

4.5 E5: NPC behaviour

Task. Twelve paired combat scenarios, each pair differing in one fact, with the expected direction stated before running (e.g. a goblin with all allies dead and an open tunnel behind it should flee more; the same goblin in a dead-end alcove should flee less; a mindless zombie should not flee). Two questions per request in one call: what the creature does (FlexAI’s six outcomes) and whom it

Policy	Pairs moving the expected way ($\Delta > 0.02$)
FlexAI d100 tables (role $\times$ stance)	5 / 12
clef	11 / 12
clef-flash	11 / 12
Decision-2.0-Lux-9B	11 / 12
Decision-2.0-Nox-4B	11 / 12
Decision-2.0-Kai-0.6B	5 / 12
qwen, fast (asked for a distribution)	9 / 12

Table 6: E5. Two pairs are imperfect: in one, clef’s answer (an archer engaged in melee draws its sword, *attack secondary* 0.64) is arguably better than the expected *maneuver*; in another, the shared party description contradicted the scenario’s premise, invalidating the pair for all policies. The scenarios and expectations were written by the experimenter.

targets (FlexAI’s eight targeting rules). FlexAI’s own d100 tables are the baseline; they see only role and stance.

Result (Table 6). Situation-aware decision models clearly beat the dice tables, and their probabilities are graded (goblin *flee* 0.03 $\rightarrow$ 0.75 when its allies die, $\rightarrow$ 0.43 when cornered), which is what a sampler needs. The generative model’s stated distributions were nearly degenerate (many 0.00 and 1.00 values, e.g. 1.00 on a single target), so sampling from them adds little variety. On this evidence we built a table-side prototype: the GM types one line describing the situation, a decision model returns stance probabilities over eight social responses and samples one, and the generative model voices a line in that stance. With Decision-2.0-Nox-4B a suggestion takes 0.25–0.8 s including the NPC’s dossier, and the line about 2.5–3 s. A labelled evaluation of social responses against 60 past moments has been prepared but not yet adjudicated by the GM; we do not report it.

4.6 E6: Serving on the GB10

Latency. Table 1 summarises it. Decision 2.0’s runtime ships custom kernels only for AMD’s gfx942; on CUDA it uses pure-PyTorch reference kernels, and its CUDA path is marked unvalidated upstream. Measured medians on the GB10 were 3–7 $\times$ the model-card figures (Kai 16 vs. 4.9 ms; Lux 127 vs. 18.4 ms). For Clef, the GB10 was 3.5 $\times$ slower than Cloudflare’s published H200 medians for the 9B model but only 1.9 $\times$ for the 27B, consistent with fixed per-request overhead dominating small models.

Correctness. With no upstream golden check on CUDA, we compared Kai-0.6B’s CUDA and CPU outputs on identical requests: probabilities agreed within 0.005.

Memory. The runtime’s placement check used `torch.cuda.mem_get_info`, which on the GB10’s unified memory excludes reclaimable page cache; the box reported 9–18 GiB “free” with 34 GB available, and reading the weights to verify checksums refilled the cache just before the check. We patched the check to use the kernel’s `MemAvailable` on integrated devices. Separately, Lux-9B occupied about 50 GB (33 GB resident in the process) for 16 GB of BF16 weights; co-hosted with Clef the system fell to 1 GB available and the runtime crashed once.

5 Discussion

What kind of label is it? Across the five tasks a single distinction predicted success. Decision models succeeded when the label is a *coarse property readable from meaning*: is this token chatter or a campaign name; would a cornered creature flee. They failed when the label is (a) a *string-level*

judgment (which spelling, exactly which words), (b) a *convention* the text does not reveal (how this GM files demon lords), or (c) a *consistency property across items* (one type per entity across many facts). Most judgment steps in this pipeline are of the second kind, which is why the decision models’ role ended up narrow.

Filters and flags, not deciders. Every useful configuration kept the human checkpoint intact: triage that orders work without deleting it, a second opinion that adds subjects to a review queue, and sampled NPC suggestions the GM reads. None of these removes a decision from the human.

Calibration has two uses. Graded probabilities mattered in two different ways: to set a zero-loss filter threshold (E3) and to sample plausible behaviour (E5). The generative baseline’s self-reported probabilities served neither purpose.

Two ways a convincing result was wrong. E1’s AUC came from the classifier reading the upstream model’s prose, and E4’s per-fact improvement came from a test set selected on errors. Both looked strong until a baseline or a control was added. We recommend both checks for any evaluation built from a pipeline’s own review records.

6 Limitations

This is a single-practitioner study: one campaign, one GM, one hardware platform. Several label sets are small (84 real names in E3; 10 non-approved proposals in E1; 12 scenario pairs in E5). Some labels are weak (“left alone” in E3), and some test material was written by the experimenter (E5). The E2 set was built from approved proposals and cannot reveal the upstream model’s misses. Prompts and option descriptions were written once and not tuned, which is fair to a drop-in classifier but unfavourable to it. We did not verify that the locally served Clef matches Cloudflare’s hosted service. Decision 2.0 ran on a runtime path its authors mark as unvalidated on CUDA; our own CPU/GPU parity check covers only the smallest model. Latencies are single-stream; the runtime’s batching profiles were not tested.

7 Conclusion

Decision models are a useful addition to a local LLM toolkit, in a narrow role: fast, calibrated classification of coarse properties, placed where it orders or flags work rather than deciding it. In this pipeline that meant triaging transcript tokens without silent losses and generating situation-aware NPC behaviour at the table. They did not replace a generative model with reasoning for spelling decisions, could not infer house conventions, and cannot impose consistency across items that are judged one at a time. On the GB10, the smaller Decision 2.0 models give most of the behavioural quality at a fraction of the memory, but published latency claims should be re-measured on the target hardware.

Data and code availability

Serving configurations, the GB10 patch, and all evaluation scripts are available in the public repository [9]. The campaign data is not released because it quotes play sessions verbatim and names real people.

Use of AI assistance

The experiments, analysis and a first draft of this paper were carried out by an AI coding agent (Claude, Anthropic) working under the author’s direction, with the author reviewing results and making the decisions recorded in the paper. The author takes full responsibility for the content.

References

- [1] Cloudflare. Clef and Clef-flash decision models. Model cards, <https://huggingface.co/Cloudflare/clef> and <https://huggingface.co/Cloudflare/clef-flash>, October 2026.
- [2] vLLM Semantic Router Team. Decision 2.0: towards open foundation decision models. Model collection, <https://huggingface.co/collections/vllm-sr/decision-20>, October 2026.
- [3] vLLM Semantic Router Team. Built-in model runtime, phase 1: serve Decision 2.0 from vllm-sr. GitHub issue #4479, <https://github.com/vllm-project/semantic-router/issues/4479>, 2026.
- [4] vLLM Semantic Router. When to reason: Semantic router for vLLM. arXiv:2510.08731, 2025.
- [5] vLLM Semantic Router. The workload-router-pool architecture for LLM inference optimization: A vision paper from the vLLM Semantic Router project. arXiv:2603.21354, 2026.
- [6] Infinium Game Studios. *FlexAI Guidebook*. 2020.
- [7] J. A. Hanley and B. J. McNeil. The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology*, 143(1):29–36, 1982.
- [8] L. Philips. The double metaphone search algorithm. *C/C++ Users Journal*, 18(6):38–43, 2000.
- [9] [Author]. dgx-fun: experiments with local LLM serving on NVIDIA DGX Spark. <https://github.com/kostadis/dgx-fun>, 2026.